

Potential Questions & Answers

Hardware/Software Co-Design for LLM Quantization
Gyanu Pandey • CECS 528 • December 2024

 **Preparation Tip:** Questions are organized by topic and difficulty. Focus on the "Hard" questions — if you can answer those confidently, the easier ones will be no problem.

1. Fundamentals — Memory Wall & Quantization (Slides 2-5)

QUESTION **EASY**

Why can't we just run LLaMA-70B on a regular GPU?

SUGGESTED ANSWER

LLaMA-70B needs 140GB just to load the weights. A consumer GPU like the RTX 4090 has 24GB. Even an A100 — which costs \$15K+ — only has 80GB. The model literally doesn't fit in memory. That's the memory wall problem.

QUESTION **EASY**

What exactly is quantization?

SUGGESTED ANSWER

Quantization reduces the numerical precision of neural network weights. Instead of using 32-bit floating point (FP32), we use 8-bit integers (INT8) or even 4-bit (INT4). This shrinks the model by 4-8x — so a 140GB model becomes 35GB at INT4.

QUESTION **MEDIUM**

Why does quantization sometimes hurt accuracy?

SUGGESTED ANSWER

When you reduce from 32 bits to 4 bits, you have far fewer possible values. If all your weights cluster around zero (which they do), that's fine. But outliers — weights that are 10-100x larger — get clipped or misrepresented. These outliers often carry important information, so losing them hurts accuracy.

QUESTION **MEDIUM**

What's the difference between memory-bound and compute-bound?

SUGGESTED ANSWER

Compute-bound means your processor can't do math fast enough. Memory-bound means your processor is fast but sits idle waiting for data. LLM inference is memory-bound — the A100 has 312 TFLOPS of compute but to generate one token, you have to read all 70 billion weights. The memory bandwidth is the bottleneck, not the compute.

QUESTION **HARD**

Why is \$700K/day for ChatGPT inference? How did they estimate that?

SUGGESTED ANSWER

It's an estimate based on: (1) Millions of daily users, (2) Each query requires multiple GPU-seconds of inference, (3) A100 clusters cost ~\$1-3 per GPU-hour including power and cooling, (4) Multiply by the estimated query volume. Different sources give different numbers (\$700K is on the higher end), but even conservative estimates are hundreds of thousands per day.

2. Software Quantization Limitations (Slides 7-8)

QUESTION EASY

What are some popular software quantization tools?

SUGGESTED ANSWER

bitsandbytes, ONNX Runtime, llama.cpp, and AutoGPTQ. They run on standard NVIDIA GPUs and don't require specialized hardware.

QUESTION MEDIUM

Why is bitsandbytes sometimes slower than running without quantization?

SUGGESTED ANSWER

Dequantization overhead. GPUs can't compute directly on INT8 weights — they have to convert INT8 → FP16, do the calculation, then convert back. This conversion happens on every operation. For small batch sizes, this overhead exceeds the memory savings. CipherCore benchmarks showed bitsandbytes at 0.65× baseline — 35% slower.

QUESTION MEDIUM

Why does ONNX Runtime perform better than bitsandbytes?

SUGGESTED ANSWER

ONNX Runtime uses kernel fusion and better memory management. It combines multiple operations into single kernels, reducing dequantization steps. It also optimizes memory layout. That's why it achieves 1.7× speedup while bitsandbytes doesn't.

QUESTION HARD

If software quantization has overhead, why do companies still use it?

SUGGESTED ANSWER

Three reasons: (1) Accessibility — just pip install, no custom hardware needed. (2) Memory reduction still matters — even if inference isn't faster, fitting the model in memory at all is valuable. (3) Different workloads — large batch sizes can still benefit. For many use cases, "slower but fits" beats "fast but won't load."

3. Oltron — ASIC Approach (Slides 10-12)

QUESTION EASY

What is Oltron and who developed it?

SUGGESTED ANSWER

Oltron is a custom ASIC (Application-Specific Integrated Circuit) developed by Peking University, published at DAC 2024. It's designed specifically for LLM inference with mixed-precision quantization.

QUESTION MEDIUM

What's the "outlier problem" that Oltron solves?

SUGGESTED ANSWER

In neural networks, 99% of weights are small, clustered around zero. But ~1% are outliers — 10-100× larger. If you scale quantization for normal values, outliers overflow. If you scale for outliers, you lose precision on everything else. You can't win with a single scale factor.

QUESTION MEDIUM

How does "adaptive quantization" work in Oltron?

SUGGESTED ANSWER

Oltron analyzes each layer's weight distribution before inference. It identifies which channels have outliers and allocates precision accordingly — some channels get INT4, others get FP16. Layer 1 might need 2% FP16 for outliers, Layer 5 might need 15%. The ratio adapts per-layer.

QUESTION HARD

What are "reconfigurable processing elements" and why do they matter?

SUGGESTED ANSWER

Unlike GPU cores that are fixed (always FP16 or always INT8), Oltron's PEs can dynamically switch between INT4 and FP16 operations at runtime. When a layer needs more FP16 for outliers, the hardware physically reconfigures to provide that. This is the "co-design" part — the hardware adapts to what the algorithm needs.

QUESTION HARD

What is "tile-based dataflow" and how does it reduce the memory bottleneck?

SUGGESTED ANSWER

Instead of processing the whole matrix at once (which requires loading everything from memory), tile-based dataflow breaks computation into small tiles. Each tile is loaded once and reused multiple times before moving to the next. This maximizes data reuse — you do more computation per memory access, reducing bandwidth requirements.

QUESTION HARD

Why did Oltron use 12nm process technology? Is that outdated?

SUGGESTED ANSWER

12nm is mature and cost-effective for academic research. Cutting-edge nodes (7nm, 5nm) cost millions for tape-out. The innovation is in the architecture, not the process node. Commercial versions could use newer nodes for even better results. The 1.9× speedup is despite using older technology.

4. FlightLLM — FPGA Approach (Slides 13-15)

QUESTION EASY

What's the difference between an FPGA and a GPU?

SUGGESTED ANSWER

A GPU has fixed architecture — thousands of identical cores doing the same operations. An FPGA is reconfigurable — you can design custom circuits for your specific workload. GPUs are like a factory with fixed assembly lines. FPGAs are like a workshop where you build custom machines.

QUESTION MEDIUM

How does FlightLLM skip zero computations?

SUGGESTED ANSWER

They built custom DSP units called "Configurable Sparse DSP." These units detect when a weight or activation is zero and skip the entire multiplication. GPUs can't do this — they process every value whether it's zero or not. FlightLLM gets 1.6× compute efficiency just from skipping zeros.

QUESTION MEDIUM

What is the KV cache and why does keeping it on-chip matter?

SUGGESTED ANSWER

The KV (Key-Value) cache stores attention states from previous tokens. During autoregressive generation, you need to access it for every new token. On GPUs, this lives in DRAM — slow. FlightLLM keeps it in SRAM on the FPGA — 100× faster. This eliminates a major memory bottleneck.

QUESTION HARD

Why does FlightLLM achieve 6× energy efficiency vs GPUs?

SUGGESTED ANSWER

Three factors: (1) Skipping zeros means less computation per token. (2) On-chip memory means less data movement — moving data is the biggest power consumer. (3) FPGAs run at lower frequencies but with custom circuits, so less wasted work. 75W vs 300W for similar throughput.

QUESTION HARD

What is "length-adaptive compilation" and why is it important?

SUGGESTED ANSWER

FPGAs normally need to be "compiled" (synthesized) for a specific configuration. If your model expects 512 tokens but gets 1024, you'd need to recompile. FlightLLM designed their architecture so one compilation works for any sequence length from 128 to 2048+. This makes deployment practical.

QUESTION **MEDIUM**

When would you choose ASIC over FPGA?

SUGGESTED ANSWER

ASICs for hyperscale production — if you're Google or OpenAI deploying millions of chips, the higher efficiency justifies the upfront cost. FPGAs for research or smaller deployments — they're cheaper, faster to develop, and you can update them as models evolve. Oltron (ASIC) can't be changed; FlightLLM (FPGA) can be reprogrammed.

QUESTION **MEDIUM**

Why is FlightLLM's 6× energy efficiency more important than Oltron's 1.9× speedup?

SUGGESTED ANSWER

At datacenter scale, power is the limiting factor. You can always add more chips for throughput, but cooling and electricity have physical limits. A 6× energy reduction means you can run 6× more inference per watt of power — that directly impacts operational cost and scalability.

QUESTION **HARD**

Both papers claim less than 1% accuracy loss. How reliable is that claim?

SUGGESTED ANSWER

It depends on the benchmark. Perplexity on WikiText is standard, but real-world tasks may differ. Also, "less than 1%" on perplexity doesn't mean all tasks are unaffected — some specific capabilities might degrade more. The papers are honest about their metrics, but real deployment would need task-specific validation.

6. Critical Thinking Questions (Professor may ask these)

QUESTION HARD

Why not just use bigger GPUs or more GPUs instead of custom hardware?

SUGGESTED ANSWER

Even the biggest GPU (H100 with 80GB) can't fit the largest models. And adding more GPUs adds communication overhead — you have to split the model across GPUs and synchronize, which kills efficiency. Custom hardware solves the fundamental bottleneck (memory bandwidth, dequantization overhead) rather than just throwing more resources at it.

QUESTION HARD

These papers are from 2024. Are they already outdated?

SUGGESTED ANSWER

The specific implementations may be superseded, but the principles remain valid: (1) Memory-bound workloads need co-designed solutions. (2) Adaptive quantization beats uniform quantization. (3) Sparsity exploitation requires hardware support. These insights apply to any future hardware — Groq, AWS Inferentia, etc. all use similar ideas.

QUESTION HARD

How does this connect to what we learned in Advanced Computer Architecture?

SUGGESTED ANSWER

Several connections: (1) Memory hierarchy — SRAM vs DRAM trade-offs, on-chip caching strategies. (2) Dataflow architecture — how data moves through processing elements, tiling strategies. (3) Amdahl's Law — focusing optimization on the bottleneck (memory, not compute). (4) Specialization vs generalization — ASICs vs FPGAs vs GPUs trade-off space.

QUESTION HARD

What would you do differently if you were to extend this research?

SUGGESTED ANSWER

Three directions: (1) Hands-on benchmarking — validate the software numbers myself with bitsandbytes and llama.cpp on actual hardware. (2) Emerging accelerators — compare against Groq LPU, AWS Inferentia, Google TPU which have native quantization support. (3) Total cost of ownership analysis — GPU rental vs FPGA vs ASIC over 3-5 year deployment.

QUESTION HARD

You mentioned CipherCore benchmarks. Did you verify them yourself?

SUGGESTED ANSWER

No, this is a literature review and comparative analysis, not an implementation project. I used CipherCore's published benchmarks. For future work, I'd want to replicate those numbers — benchmark results can vary significantly based on batch size, sequence length, GPU generation, and software versions.

7. Potential "Gotcha" Questions

QUESTION HARD

Isn't NVIDIA working on native INT4 support? Does that make this research obsolete?

SUGGESTED ANSWER

NVIDIA's Hopper (H100) and Ada architectures do have improved INT8 support, and there are FP8 tensor cores. But they still can't: (1) Skip zeros dynamically, (2) Adapt precision per-channel at runtime, (3) Eliminate all dequantization overhead. The gap is smaller, but custom hardware still wins on efficiency.

QUESTION HARD

FlightLLM uses a \$5K FPGA. How does that compare to cloud GPU costs?

SUGGESTED ANSWER

A100 cloud instances cost ~\$1-4 per hour. At \$2/hour, the \$5K FPGA pays for itself in 2,500 hours (~104 days of continuous use). But FPGAs need development time and expertise. For short-term or variable workloads, cloud GPUs make sense. For steady-state production, custom hardware wins on TCO.

QUESTION MEDIUM

What if the model doesn't have many zeros? Does FlightLLM's sparse DSP still help?

SUGGESTED ANSWER

Good question. Sparse DSP helps most with quantized models where zeros are common (INT4 often has 30-50% zeros). Dense FP16 models have fewer exact zeros. However, FlightLLM's other innovations (on-chip KV cache, length-adaptive compile) still help regardless of sparsity.

Final Tips:

- If you don't know an answer, say "That's a great question — I'd need to look deeper into that." Don't make things up.
- Circle back to the key numbers: 140GB, 1.9x, 6x, 75W vs 300W.
- If asked about limitations, be honest: "This is a literature review. I didn't implement anything myself."